arXiv:2108.00708v1 [cs.CV] 2 Aug 2021

Group Fisher Pruning for Practical Network Compression

Liyang Liu [*1] **Shilong Zhang** [*2] **Zhanghui Kuang** [3] **Aojun Zhou** [3] **Jing-Hao Xue** [4]
Xinjiang Wang [3] **Yimin Chen** [3] **Wenming Yang** [1] **Qingmin Liao** [1] **Wayne Zhang** [2 3 5]

Abstract

Network compression has been widely studied since it is able to reduce the memory and computation cost during inference. However, previous methods seldom deal with complicated structures like residual connections, group/depth-wise convolution and feature pyramid network, where channels of multiple layers are coupled and need to be pruned simultaneously. In this paper, we present a general channel pruning approach that can be applied to various complicated structures. Particularly, we propose a layer grouping algorithm to find coupled channels automatically. Then we derive a unified metric based on Fisher information to evaluate the importance of a single channel and coupled channels. Moreover, we find that inference speedup on GPUs is more correlated with the reduction of memory[6] rather than FLOPs, and thus we employ the memory reduction of each channel to normalize the importance. Our method can be used to prune any structures including those with coupled channels. We conduct extensive experiments on various backbones, including the classic ResNet and ResNeXt, mobile-friendly MobileNetV2, and the NAS-based RegNet, both on image classification and object detection which is under-explored. Experimental results validate that our method can effectively prune sophisticated networks, boosting inference speed without sacrificing accuracy.

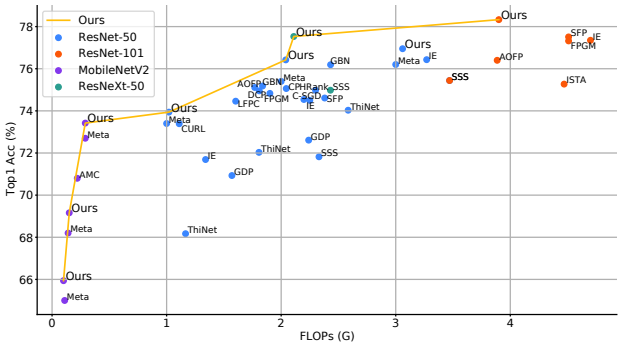


Figure 1: Compare top-1 accuracies of our pruned models with state-of-the-arts under various FLOPs and network structures.

1. Introduction

Modern computer vision models equipped with deep networks exhibit excellent performances in many tasks. However, they consume a great amount of memory and computation during inference. It can hinder the model deployment on edge devices where high-end hardwares are not available. It can also limit the throughput of services on clouds, resulting from considerable energy cost and inference latency. Network pruning aims at increasing the inference efficiency with negligible accuracy drop. It takes the trained dense model as input and prunes weights or channels with little importances. Through fine-tuning the pruned model can usually regain the lost performance caused by pruning.

Although numerous *channel pruning* methods have been proposed in literature (Molchanov et al., 2017; Luo et al., 2017; He et al., 2017; Liu et al., 2017), most of them study sequential networks such as AlexNet (Krizhevsky et al., 2012) and VGGNet (Simonyan & Zisserman, 2015) where pruning the input channel of a layer only affects the output channel of its single preceding layer. However, recently developed networks are designed with complicated structures such as residual connections in ResNet (He et al., 2016), group convolution (GConv) in ResNeXt (Xie et al., 2017) and RegNet (Radosavovic et al., 2020), depth-wise convolution (DWConv) in MobileNet (Sandler et al., 2018), and feature pyramid networks (FPN) (Lin et al., 2017a) in object detection frameworks. These structures have *coupled channels* distributed in multiple layers, which must be pruned or

[*]Equal contribution [1]Shenzhen International Graduate School/Department of Electronic Engineering, Tsinghua University, Beijing, China [2]Shanghai AI Laboratory, Shanghai, China [3]SenseTime Research, Hong Kong, China [4]Department of Statistical Science, University College London, London, United Kingdom [5]Qing Yuan Research Institute, Shanghai, China. Correspondence to: Wenming Yang <yang.wenming@sz.tsinghua.edu.cn>.

Proceedings of the 38th International Conference on Machine Learning, PMLR 139, 2021. Copyright 2021 by the author(s).

[6]We use "memory" to denote the number of elements in the output feature maps of all layers. The code will be available at https://github.com/jshilong/FisherPruning.

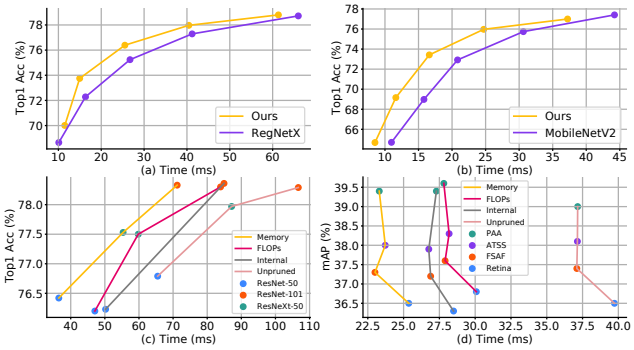


Figure 2: Efficiency of our proposed method. In (a) and (b) we compare our pruned models with the searched RegNet and uniform-scaled MobileNetV2, respectively. In (c) and (d) *circles* with different colors represent different network structures, and *lines* show the results of pruning by different strategies. "Memory" is our full method which normalizes importance scores by memory reduction and prunes coupled channels via layer grouping. "Internal" only prunes the isolated channels. "FLOPs" normalizes importances by FLOPs. "Unpruned" represents the unpruned model. In (d) different detection networks are pruned and compared with the baselines. The x-axis shows the inference time.

preserved simultaneously. Ignoring the coupled channels and pruning them independently will definitely hurt the efficiency in terms of both FLOPs (floating-point operations), memory access and actual speedup during inference.

In this paper we propose a general framework named *Group Fisher Pruning* that can be applied to various complicated structures. Particularly, we first introduce a binary mask initialized as 1 for each input channel. Then we propose a *layer grouping* algorithm to automatically find the coupled channels given computation graph of the network, and we make the coupled channels *share* the same mask. The importance of a single channel is estimated by the loss change if it is discarded, which is approximated by Fisher information and is proportional to the squared mask gradient. Based on the single-channel importance, we obtain the overall importance of coupled channels by the principled chain rule of gradient computation. Pruning is done by iteratively setting the mask of the least important channel to 0, where the coupled channels are pruned together. Finally the network is fine-tuned to regain the lost accuracy. During fine-tuning and inference, the channels with 0 masks are explicitly excluded from the network, and thus computation and memory cost can be practically reduced for acceleration.

Moreover, we propose to normalize importances of channels by their reductions of computation costs as we would like to prune the least important channels with the most computation overheads to achieve the best trade-off between accuracy and efficiency. However, we find the commonly-used reduction of FLOPs is a rather biased estimator for the actual inference speedup. In contrast, we propose to measure the computational complexity of a channel by its reduction of memory during pruning. Through experiments we find normalizing the channel importance by the reduction of memory is more correlated with the speedup than FLOPs in terms of the inference time on GPUs.

Our proposed Group Fisher Pruning has the following advantages. **Firstly**, it can prune any layers including those with coupled channels, and thus achieves better trade-off between accuracy drop and actual acceleration. **Secondly**, it prunes globally rather than locally (He et al., 2017; Luo et al., 2017), *i.e.*, it obtains the pruning ratio for each layer automatically without the cumbersome sensitivity analysis of layer-wise pruning ratio (Yu et al., 2018), and thus leads to higher accuracy. **Thirdly**, it estimates importances of all channels in one pass via the principled Fisher information instead of multiple forward passes for individual channels (Luo & Wu, 2020), and thus is more efficient. **Lastly**, in contrast with (Liu et al., 2017), it does not depend on specific layers like batch normalization (BN) and thus is more general so that we can prune more sophisticated structures such as object detection networks, where such layers may not be naïvely adopted due to the larger input size.

To demonstrate the generalization ability and effectiveness of the proposed method to deal with complicated network structures, we conduct extensive experiments on various backbones, including classic ResNet (He et al., 2016) and ResNeXt (Xie et al., 2017), mobile-friendly MobileNetV2 (Sandler et al., 2018), and recent NAS-based RegNet (Radosavovic et al., 2020) on image classification (See Fig. 1). We also evaluate our method on object detection, which is more computation-intensive due to larger input image size and more complicated network structure than image classification, but rather under-explored (See Fig. 2).

Our main contributions are: we introduce the concept of coupled channels, find them by the proposed layer grouping algorithm, derive a unified metric to evaluate both coupled-channel and single-channel importances (based on Fisher information), and normalize the importance by memory reduction to realize higher speedup on GPUs without sacrificing the accuracy.

2. Related Work

Network pruning can be generally categorized into unstructured and structured methods. Unstructured methods (Han et al., 2015; 2016; Guo et al., 2016) prune unimportant weights in the model, but efficiency of the pruned sparse network can only be shown with the help of specialized libraries or hardwares. Recently, there are also efforts (Zhou et al., 2021; Mishra et al., 2021) to develop N:M fine-grained sparse models, leveraging the innovations in general-

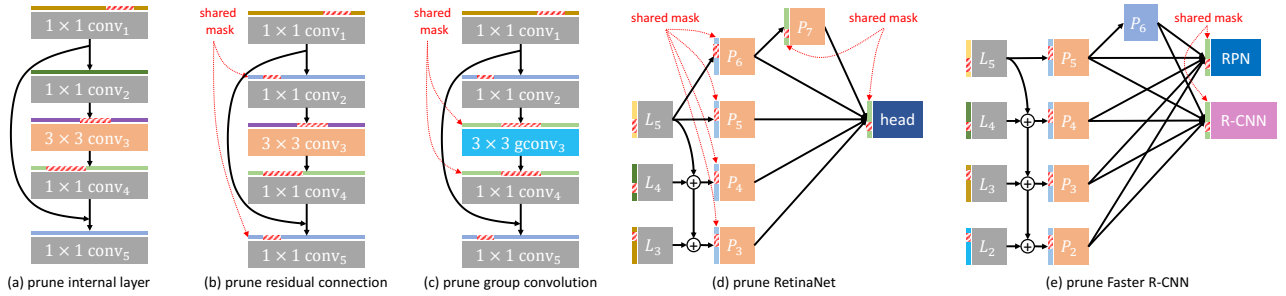


Figure 3: Prune different constrained structures. For each Conv/FC layer we introduce channel-wise binary input masks, where masks with the same color are shared and red stripes show the pruned channels. Many previous methods only prune internal layers in a residual block when residual connections exist as in (a), which will lead to lower efficiency since only *output channels* of conv_2/conv_3 can be pruned but not those of conv_1/conv_4. In contrast, we find coupled channels *automatically* by our proposed layer grouping algorithm and make them *share* masks as in (b), where conv_2 and conv_5 are found to have coupled channels as they both are children of conv_1 in the computation graph. We prune the coupled channels together such that the output channels of conv_1/conv_4 can also be pruned correspondingly, leading to higher efficiency. In (c) except for mask sharing in conv_2/conv_5, the input and output channels of group convolution (gconv) are also coupled and should be pruned simultaneously, which results in the mask sharing of conv_3/conv_4. In (d) and (e) feature pyramid and head networks for one-stage and two-stage detection frameworks are shown, where the structures are more complicated than classification networks. Note that P_6 in (e) is a pooling layer rather than Conv layer, so there is no mask assigned to it. Even though, our method is still able to achieve high efficiency via mask sharing for coupled channels.

purpose GPUs (*e.g.*, NVIDIA Ampere architecture). On the contrary, structured methods prune the whole channels or filters with little importances, and thus actual speedup can be easily achieved without requiring sparse accelerators.

For structured pruning methods (Wen et al., 2016; Lebedev & Lempitsky, 2016), different importance metrics have been proposed. PFEC (Li et al., 2017) employs L_1 norm of the channel weights, while SFP (He et al., 2018a) uses L_2 norm of each filter. These methods rely on the "smaller-norm-less-informative" assumption (Ye et al., 2018) which may not be true especially for structured pruning. CP (He et al., 2017) and ThiNet (Luo et al., 2017) cast channel selection as reconstruction error minimization of feature maps, where LASSO regression and greedy strategy are used to select the pruned channels, respectively. However, they can only prune networks in a layer-wise manner, as the least-square reconstruction happens locally. NISP (Yu et al., 2018) instead minimizes the reconstruction error of the final response layer and propagates importance scores through the entire network. The above methods also need sensitivity analysis to decide the pruning ratio for each layer, which may be time-consuming and sub-optimal. Network Slimming (Liu et al., 2017) reuses BN layer scaling factors as importance scores so that channels can be pruned globally. Although BN is prevalently used in image classification, many applications such as object detection can not trivially adopt BN because of the large input image size, which limits the application scenarios of pruning methods based on BN scaling factors. SSS (Huang & Wang, 2018) introduces extra scaling factors to scale the outputs of various micro-structures and solves the sparsity regularized optimization of scaling factors by the accelerated proximal gradient method.

Apart from the heuristic-based importance evaluation methods, one may use the exact loss change induced by removing a specific parameter (Luo & Wu, 2020) to measure its importance, but it is prohibitively expensive due to the large parameter number. Others try to approximate the importance score via Taylor expansion on the loss. The seminal work of OBD (LeCun et al., 1990) and OBS (Hassibi & Stork, 1993) exploit the second-order derivative information to estimate the importances of weights, but they may need to obtain the heavy-weight Hessian matrix which is too large to compute for modern large-scale networks. L-OBS (Dong et al., 2017) layer-wisely computes the Hessian matrix to achieve tractable approximation. WoodFisher (Singh & Alistarh, 2020) approximates the inverse of Hessian matrix by the Woodbury matrix identity and improves unstructured pruning based on OBD/OBS. PCNN (Molchanov et al., 2017) extends Taylor expansion to channel pruning and uses the first-order information instead. It takes a greedy strategy to prune the least important channels, interleaving pruning and fine-tuning. In place of estimating importances of feature maps, IE (Molchanov et al., 2019) applies Taylor expansion to the weights of a filter. These importance estimation methods are more principled than magnitude-based ones, but they seldom deal with structure constraints, for example, the residual connections.

Besides the importances, another factor needed to be concerned is computation, as we wish to prune the least important channels with the most computation costs. Current methods (Molchanov et al., 2017; Theis et al., 2018) typi-

cally add a regularization term to constrain FLOPs of the pruned network. However, the same amount of FLOPs reduction may lead to different actual speedups. Through experiments we empirically find that reduction of memory access can act as a more accurate estimator for efficiency gain, which is not explored in previous pruning methods.

3. Methodology

We first introduce Fisher information (Theis et al., 2018) as single-channel importance estimation, which can be used to prune channels in sequential networks but can not deal with complicated structures. Then we propose our layer grouping algorithm to find coupled channels in different layers, and make the coupled channels share the mask so as to prune them simultaneously. Finally we propose to use memory reduction as importance normalization to achieve better trade-off between efficiency and accuracy.

Given a training dataset $\mathcal{D} = \{\boldsymbol{x}_n, \boldsymbol{y}_n\}$ of image-label pairs and a network $\boldsymbol{W}_0$ trained on it to convergence, we aim to prune the least important channel. We introduce a binary mask (initialized as 1) for each input channel to achieve structured pruning, and one channel can be pruned by setting its mask to 0. During pruning, the input tensor $\mathbf{A} \in \mathbb{R}^{n\times c\times h\times w}$ for a convolution (Conv) or fully-connected (FC) layer ($h = w = 1$ in FC) is element-wisely multiplied by the masks $\boldsymbol{m} \in \mathbb{R}^c$ with broadcasting to form the masked input $\widetilde{\mathbf{A}} = \mathbf{A} \odot \boldsymbol{m}$, which is the actual input for each layer. During inference we explicitly discard the channels with 0 masks, both for a layer and its parents in the computation graph, since pruning input channels of one layer also prunes output channels of its preceding layers.

3.1. Fisher Information Importance

To evaluate the importance s_i of a channel i, we apply Taylor expansion to the network loss $\mathcal{L}$ and approximate the loss change when discarding it (setting its mask to 0):

$$
\begin{aligned}
s_i &= \mathcal{L}\left(\boldsymbol{m} - \boldsymbol{e}_i\right) - \mathcal{L}\left(\boldsymbol{m}\right) \approx -\boldsymbol{e}_i^\top \nabla_{\boldsymbol{m}} \mathcal{L} + \frac{1}{2} \boldsymbol{e}_i^\top \left(\nabla^2_{\boldsymbol{m}} \mathcal{L}\right) \boldsymbol{e}_i \\
&= -\boldsymbol{e}_i^\top \boldsymbol{g} + \frac{1}{2} \boldsymbol{e}_i^\top \boldsymbol{H} \boldsymbol{e}_i = -g_i + \frac{1}{2} H_{ii}, \qquad (1)
\end{aligned}
$$

$\boldsymbol{m} = \mathbf{1}$ is the all-one vector and $\boldsymbol{e}_i \in \mathbb{R}^c$ denotes the one-hot vector of which the i-th element equals 1. $\boldsymbol{g} \in \mathbb{R}^c$ is the gradient *w.r.t.* $\boldsymbol{m}$ and $\boldsymbol{H} \in \mathbb{R}^{c\times c}$ is the Hessian matrix. As the model has converged and recall that $\widetilde{\mathbf{A}} = \mathbf{A} \odot \boldsymbol{m}$, $\boldsymbol{m} = \mathbf{1}$ before pruning, we have $\nabla_{\widetilde{\mathbf{A}}} \mathcal{L} = \nabla_{\mathbf{A}} \mathcal{L} \approx \mathbf{0}$. Then we can obtain $\boldsymbol{g} = \nabla_{\boldsymbol{m}} \mathcal{L} \approx \mathbf{0}$, *i.e.*, $g_i = \frac{\partial \mathcal{L}}{\partial m_i} = \frac{1}{N} \sum \frac{\partial \mathcal{L}_n}{\partial m_i} \approx 0$ where $\mathcal{L}_n$ denotes the negative log-likelihood loss of n-th sample and $\frac{\partial \mathcal{L}_n}{\partial m_i}$ is the sample-wise gradient. We compute the diagonal entry H_{ii} of the Hessian matrix $\boldsymbol{H}$:

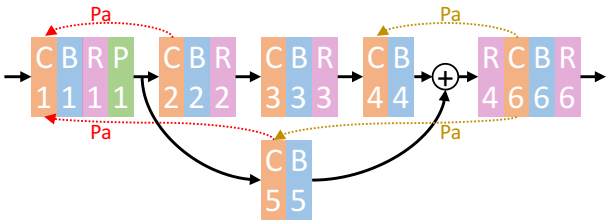


Figure 4: Depth-first search for finding parents (Pa). We take the first residual block of ResNet-50 as an example which consists of Conv (C), BN (B), ReLU (R) and Pooling (P) layers. During layer grouping we ignore all layers except Conv layers, so we can adopt DFS to find that C1 has two children C2/C5, and C6 has two parents C4/C5. C2 and C5 have coupled channels which should be pruned together such that output channels of C1 can be pruned.

$$
\begin{aligned}
H_{ii} &= \frac{\partial^2 \mathcal{L}}{\partial m_i^2} = \frac{1}{N} \sum_{n=1}^{N} \frac{\partial^2 \mathcal{L}_n}{\partial m_i^2} \approx -\frac{\partial^2}{\partial m_i^2} \mathbb{E}\left[\log p\left(\boldsymbol{y} \mid \boldsymbol{x}\right)\right] \\
&= \mathbb{E}\left[-\frac{\partial}{\partial m_i} \log p\left(\boldsymbol{y} \mid \boldsymbol{x}\right)\right]^2 \approx \frac{1}{N} \sum_{n=1}^{N} \left(\frac{\partial \mathcal{L}_n}{\partial m_i}\right)^2, \qquad (2)
\end{aligned}
$$

where we employ Fisher information to transform second-order derivative to the square of first-order derivative. Note that $g_i \approx 0$ and $H_{ii} \geq 0$ which correspond to mean and variance of the sample-wise gradient, respectively. Assume $\mathbf{A}_n \in \mathbb{R}^{c\times h\times w}$ is the feature map of n-th example, we have the masked feature $\widetilde{\mathbf{A}}_n = \mathbf{A}_n \odot \widetilde{\boldsymbol{m}}$ where $\widetilde{\boldsymbol{m}} \in \mathbb{R}^{c\times h\times w}$ is broadcasted by $\boldsymbol{m} \in \mathbb{R}^c$, so we can compute $\nabla_{\widetilde{\boldsymbol{m}}} \mathcal{L}_n = \mathbf{A}_n \odot \nabla_{\widetilde{\mathbf{A}}_n} \mathcal{L}_n \in \mathbb{R}^{c\times h\times w}$, in which $\nabla_{\widetilde{\mathbf{A}}_n} \mathcal{L}_n$ is already available during the backward pass without requiring additional computation. Then the sample-wise gradient *w.r.t.* $\boldsymbol{m}$ can be obtained by summing over the *spatial* dimension h and w: $\nabla_{\boldsymbol{m}} \mathcal{L}_n = \text{sum}(\nabla_{\widetilde{\boldsymbol{m}}} \mathcal{L}_n) \in \mathbb{R}^c$. Lastly the importance score for a channel can be computed by averaging the sample-wise gradients:

$$
s_i = \frac{1}{2N} \sum_{n=1}^{N} \left(\frac{\partial \mathcal{L}_n}{\partial m_i}\right)^2 \propto \sum_{n=1}^{N} \left(\frac{\partial \mathcal{L}_n}{\partial m_i}\right)^2, \qquad (3)
$$

which is proportional to the squared gradient of the mask. The above derivation is based on the model convergence, to satisfy it, a greedy pruning strategy is employed. Starting from a dense model, we first accumulate the importance scores by passing a few batches, then the least important channel is pruned. Next we fine-tune the pruned model and meanwhile re-accumulate the importance scores of the remained channels, following which the remained least important one is pruned, and the procedure recurs.

3.2. Prune Coupled Channels

Till now we can prune early-stage networks like AlexNet (Krizhevsky et al., 2012) and VGGNet (Simonyan & Zisserman, 2015) which involve normal Conv layers and sequential structures where pruning only affects a layer and its *single* preceding one. However, recent networks contain complicated structures such as residual connections (He et al., 2016), group convolutions (GConv) (Xie et al., 2017), depth-wise convolutions (DWConv) (Sandler et al., 2018) and feature pyramid networks (FPN) (Lin et al., 2017a) in object detection. There emerge coupled channels which should be pruned simultaneously to achieve higher speedup than pruning only the isolated channels. We propose mask sharing in coupled channels. Given the network computation graph $\mathcal{G}$ containing nodes like convolution (Conv), batch normalization (BN), ReLU and pooling (Pool) layers, we adopt the proposed layer grouping algorithm to find the coupled channels as Alg. 1. Firstly, we use depth-first search (DFS) as Fig. 4 to find parents $P[l_i]$ of each Conv/FC layer l_i. Since channel pruning only affects the channel dimension, we ignore all layers except Conv/FC layers in $\mathcal{G}$ during layer grouping. Then given parents of each layer, we can assign layers to different groups where layers in one group have coupled channels to be pruned simultaneously. It contains the following situations: (1) layers which have the same parents should be assigned to one group because their input channels (or equivalently, output channels of their parents) are coupled and should be pruned together as Fig. 3 (b); (2) layers whose parents contain GConv should be in the same group with their parents because the input and output channels of GConv are coupled as Fig. 3 (c). For the isolated channels, there is only one layer in the group such as the 3×3 Conv of a residual bottleneck as Fig. 3 (a).

After obtaining the coupled channels via layer grouping, we make them share the same mask. Then the overall contribution of coupled channels can be computed by:

$$s_i \propto \sum_{n=1}^{N} \left(\sum_{x \in \mathbb{X}} \frac{\partial \mathcal{L}_n}{\partial m_i^x} \frac{\partial m_i^x}{\partial m_i} \right)^2 = \sum_{n=1}^{N} \left(\sum_{x \in \mathbb{X}} \frac{\partial \mathcal{L}_n}{\partial m_i^x} \right)^2, \quad (4)$$

where m_i^x is a copy of m_i in channel x, one of the coupled channels in $\mathbb{X}$ that share the same mask m_i. The overall importance of coupled channels exactly follows the chain rule of gradient computation for shared parameters, and thus is principled without introducing any heuristics. $\mathbb{X}$ can be channels in the same layer or those distributed in different layers, and thus summation over $\mathbb{X}$ can be both *in-layer* and *cross-layer*. For the in-layer case, $\mathbb{X}$ contains channels in one layer (such as the coupled channels in i-th group of a single GConv). For the cross-layer case (such as residual connections), $\mathbb{X}$ consists of the i-th channel from all layers found by our layer grouping algorithm.

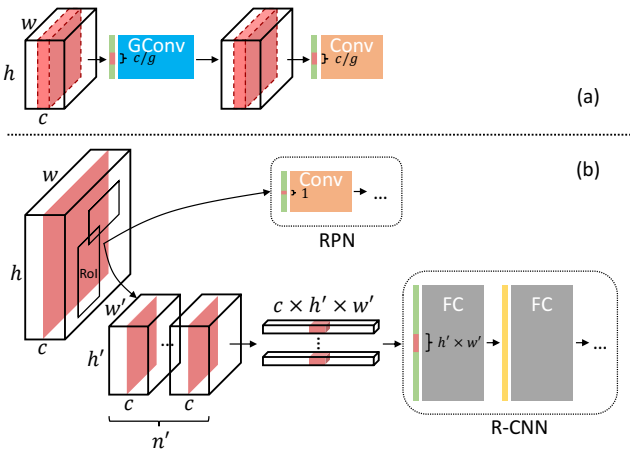


Figure 5: (a) prune group convolution (GConv). (b) prune RPN and R-CNN in Faster R-CNN.

Algorithm 1 Layer Grouping

Input: computational graph $\mathcal{G}$ with layers $\mathbb{L} = \{l_i\}$
Output: groups of layers $\mathbb{G} = \{g_i\}$ where $g_i = \{l_j\}$

for l **in** layers $\mathbb{L}$; **do**
  find its parents $P[l]$ via DFS on graph $\mathcal{G}$
end for
initialization: groups $\mathbb{G} \leftarrow \emptyset$
for l **in** layers $\mathbb{L}$; **do**
  `new_group` $\leftarrow$ True
  for g **in** groups $\mathbb{G}$; **do**
    $C \leftarrow \{l' \mid l' \in g$ and l' is GConv/DWConv$\}$
    if $P[l] \cap (P[g] \cup C) \neq \emptyset$; **then**
      $g \leftarrow g \cup \{l\}$, $P[g] \leftarrow P[g] \cup P[l]$
      `new_group` $\leftarrow$ False; **break**
  end for
  if `new_group`; **then**
    $g' \leftarrow \{l\}$, $P[g'] \leftarrow P[l]$, $\mathbb{G} \leftarrow \mathbb{G} \cup \{g'\}$
end for

Algorithm 2 Group Fisher Pruning

Input: unpruned model $\boldsymbol{W}_0$, prune interval d, training data
Output: pruned model $\boldsymbol{W}$, channel masks $\boldsymbol{m}$

initialization: $\boldsymbol{W} \leftarrow \boldsymbol{W}_0$, $\boldsymbol{m} \leftarrow \mathbf{1}$, $t \leftarrow 0$
find layers having coupled channels via Alg. 1
repeat
  forward: compute loss $\mathcal{L}$
  backward: compute gradients of parameters $\boldsymbol{W}$ and accumulate *memory-normalized* importance scores
  update model parameters $\boldsymbol{W}$ by gradient descent
  update iteration index $t \leftarrow t + 1$
  if $t \% d = 0$; **then**
    prune the least important channel i by setting $m_i = 0$
    zeroize accumulated importance scores
until remained FLOPs reduces to desired amount

For pruning **GConv** with c input channels divided into g groups as Fig. 3 (c), each time we prune one *group of channels* which share the same mask, since generally they represent related features. We first compute the c-dim gradient corresponding to c *individual* channels and reshape it to $g \times \frac{c}{g}$ (see Fig. 5 (a)), and do *in-layer* sum over the last dimension to obtain the g-dim gradient corresponding to g *groups* of channels. Next we compute the *cross-layer* summation across layers: the GConv layer itself and layers which are in the same group with the GConv including its children layers. Then we obtain the overall importance with the squared gradients as Eq. (4).

For pruning Faster R-CNN, where the first **Conv** layer of RPN and the first **FC** layer of R-CNN are assigned to the same group as Fig. 3 (e), similar computation can be done. Assume the RoIAlign layer is applied to the image-level feature map $\mathbf{A}_n \in \mathbb{R}^{c\times h\times w}$ and produces RoI features $\mathbf{F} \in \mathbb{R}^{n'\times c\times h'\times w'}$ from n' RoIs in each image (see Fig. 5 (b)). Recall that in Faster R-CNN the RoI features will be flattened as $\mathbf{F}' \in \mathbb{R}^{n'\times\left(c\times h'\times w'\right)}$ and sent to the first FC layer of R-CNN. For FC layer we can compute gradients of its masks with shape of $n' \times (c \times h' \times w')$, which is then transposed to $c \times (n' \times h' \times w')$ and *in-layer* summed over the last dimension to obtain the c-dim gradient. Now it has the same shape as gradients computed from the first Conv layer in RPN. The overall gradients of coupled channels from the RPN Conv layer and the R-CNN FC layer can be obtained via *cross-layer* summation. Lastly the gradients are used to compute the importances as Eq. (4).

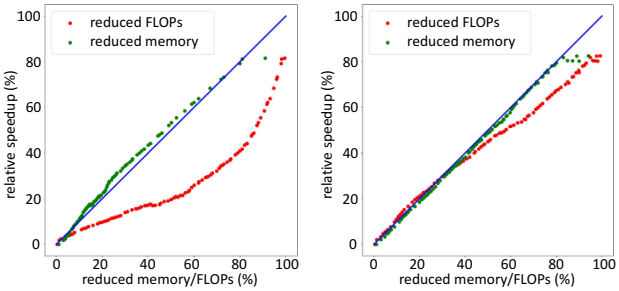
3.3. Importance Normalization

The raw importance scores do not take into consideration the computation costs of different channels, however it is more effective to prune the least important channel with the highest cost. Otherwise we may prune too many channels to achieve the desired speedup, but it may lead to degraded accuracy resulting from less parameters retained. We propose to normalize the importance scores by the computation reduction of each channel. We first try the widely-used FLOPs proxy and normalize the importance by the reduction of FLOPs $\triangle C/\triangle C_g$ for pruning an *input* channel of normal Conv/GConv as $\triangle C = n \times c'_o \times h \times w \times k_h \times k_w$ and $\triangle C_g = n \times \frac{c_o}{g} \times h \times w \times \frac{c_i}{g} \times k_h \times k_w$, where k_h/k_w denotes the kernel height/width and g is the group number in GConv. Different from previous methods which compute channel FLOPs in advance and fix them during pruning, we dynamically update the FLOPs by remained channels of the network. We use $c'_i \leq c_i$ and $c'_o \leq c_o$ to represent the unpruned input and output channel number of each layer. As the layers are connected internally, pruning a channel not only brings FLOPs reduction in the current layer, but also in its parent layers across the computation graph, which can be computed as $\triangle C^p = n \times h \times w \times c'_i \times k_h \times k_w$ and $\triangle C^p_g = n \times \frac{c_o}{g} \times h \times w \times c'_i \times k_h \times k_w$.

However, we find that reduction of FLOPs is not directly correlated with the inference speedup. In contrast, reduction of memory increases linearly with speedup (Fig. 6 (a)), which motivates us to employ the memory reduction as the importance normalization. The memory reduction of pruning one channel can be obtained for normal Conv and GConv as $\triangle M = n \times h \times w$ and $\triangle M_g = n \times \frac{c}{g} \times h \times w$. Note that we discard one group at a time when pruning GConv. Similar to FLOPs reduction, pruning an input channel in one layer brings memory reduction from all of its parents (which can be found by DFS as Fig. 4), and we obtain the overall reduction by summing separate values. Finally we adopt the memory-normalized importance $s_i/\triangle M$ of each channel to evaluate its significance and prune the least important one every few iterations. There exist methods (Wang et al., 2020; Li et al., 2020) that directly profile the running time without resorting to proxies. However, it is not applicable here since we prune in a fine-grained manner. The running time difference of discarding one or few channels is too subtle to measure. Through memory-normalization (Fig. 6 (b)), we notice that in the pruned model, the reduction of memory is still linearly correlated with speedup. Besides, the reduction of FLOPs is more correlated with speedup than the FLOPs-normalization (Fig. 6 (a)) variant. In the appendix we demonstrate that the memory is a good proxy generally applicable to various networks.

4. Experiments

In this section we first conduct ablation studies to verify the effectiveness of our layer grouping and mask sharing strategy to prune coupled channels, and that of the proposed memory normalized importance scores. We measure the batch inference time on NVIDIA 2080 Ti GPU to prove our pruned models can significantly accelerate inference with little accuracy drop. Next we show that our method can outperform previous methods to prune various networks under different FLOPs constraints including the rather compact ResNet (He et al., 2016) and ResNeXt (Xie et al., 2017) where residual connection and GConv is adopted. Our method can be applied to prune MobileNetV2 (Sandler et al., 2018) where DWConv is presented, and it outperforms the uniform-scaled baselines remarkably. It can also be used to prune RegNet (Radosavovic et al., 2020) which is neural architecture search based and highly efficient and accurate, surprisingly we achieve higher accuracy and speed than the searched counterpart under the same FLOPs. Finally, we prune object detection networks with sophisticated structures and show significant speedup with negligible mAP drop. We conduct all experiments for the task of image classification and object detection on the ImageNet (Deng et al., 2009) and COCO (Lin et al., 2014) datasets, respec-



(a) Normalize by FLOPs. (b) Normalize by memory.

Figure 6: The comparison between normalizing importances by FLOPs and memory. States of the model (reduced FLOPs/memory and relative speedup) during the pruning process are shown.

tively. We prune a channel every $d = 25/10$ iterations when pruning classification/detection networks. After the whole pruning process we fine-tune the pruned model for the same number of epochs that is used to train the unpruned model, which is trained following standard practices. The complete pruning pipeline of our proposed method is in Alg. 2. All experiments are done using PyTorch (Paszke et al., 2019) and more details can be found in the appendix.

4.1. Ablation Studies

As shown in Tab. 1, pruning coupled layers simultaneously ("-M") as Fig. 3 (b) rather than the isolated channels only ("-I") as Fig. 3 (a), for residual networks with both 50 and 101 layers, we achieve higher or comparable top-1 accuracy but with much higher inference speed, which verifies we can obtain better actual speedup under the same FLOPs. We also adopt the absolute value of the first-order gradient as the importance metric, but for pruning ResNet-50 it only obtains 75.8% top-1 accuracy, which lags behind our method based on Fisher information 76.4%.

Besides, we explore different importance normalization strategies: unnormalized ("-U"), normalized by FLOPs reduction ("-F") and normalized by memory reduction ("-M") in Fig. 2 (c) and Tab. 1. We find that normalizing importance scores by memory reduction can achieve the best accuracy-efficiency trade-off compared with the other two variants. The unnormalized importance score brings the worst efficiency gain and the largest accuracy drop, which results from the least parameter remained. For ResNet, ResNeXt and MobileNet, normalization by memory reduction is more efficiency-friendly and with higher accuracy.

Except for the rather compact residual networks, we also prune the light-weight networks MobileNetV2 (MBv2). In Fig. 2 (b) and Tab. 2 we compare the accuracy and speed of our pruned networks and the uniform scaled ones under different FLOPs budgets. To obtain a network with similar FLOPs as MBv2 (*e.g.*, MBv2-0.7×), we prune a uniform-scaled double-FLOPs MBv2 (*e.g.*, MBv2-1.0×) to 50% FLOPs remained. It can be seen that our pruned networks significantly outperform the uniform-scaled baselines. Other than only pruning the human-designed networks, we prune the highly-efficient RegNet to show that we can prune it to further boost the efficiency and accuracy. We prune RegNet (*e.g.*, RegX-1.6G) to 50% FLOPs remained to compare with the searched half-FLOPs RegNet (*e.g.*, RegX-0.8G). As shown in Fig. 2 (a) and Tab. 3, our pruned networks outperform the searched counterparts in this extreme circumstance.

Table 1: Prune ResNet, ResNeXt and MobileNetV2 on ImageNet. The **column** "T1" represents top-1 accuracy on the validation set, "F" denotes FLOPs, "P" is number of parameters, "M" means memory, "T" is the inference time and "S" shows the speedup on GPUs. The **row** with no suffix represents the unpruned model, "-I" shows the results of pruning only the internal layers, "-M" is our full method which normalizes importance scores by memory reduction and prunes coupled channels via layer grouping, "-F" normalizes the importance scores with FLOPs reduction and "-U" uses the raw Fisher information without normalization.

model	T1(%)	F(10^9)	P(10^6)	M(10^6)	T(ms)	S(×)
Res50	76.79	4.089	25.55	11.11	65.48	-
Res50-I	76.23	2.044	19.96	9.24	50.22	1.30
Res50-M	76.42	2.044	19.42	5.82	36.50	1.79
Res50-F	76.20	2.043	17.17	8.27	47.07	1.39
Res50-U	74.93	2.044	8.881	9.27	50.06	1.31
Res101	78.29	7.801	44.54	16.23	106.7	-
Res101-I	78.36	3.900	28.04	14.14	84.93	1.26
Res101-M	78.33	3.900	28.02	10.84	71.20	1.50
Res101-F	78.30	3.899	26.89	14.13	83.91	1.27
Res101-U	78.14	3.900	24.25	14.49	84.10	1.27
NeXt50	77.97	4.230	25.02	14.40	87.16	-
NeXt50-M	77.53	2.115	18.05	9.02	55.35	1.57
NeXt50-F	77.50	2.114	13.10	10.57	59.79	1.46
NeXt50-U	76.97	2.113	8.426	11.17	60.23	1.45
MBv2	77.41	1.137	11.25	13.35	44.21	-
MBv2-M	75.97	0.568	6.05	6.84	24.72	1.79
MBv2-F	75.97	0.566	5.31	11.08	37.58	1.18
MBv2-U	72.94	0.569	2.27	9.819	31.78	1.39

Table 2: Prune MobileNetV2 on ImageNet.

model	T1 (%)	F (10^9)	P (10^6)	M (10^6)	T (ms)
MBv2-2.0×	77.41	1.14	11.25	13.35	44.21
Ours	77.00	1.09	11.06	9.66	37.22
MBv2-1.4×	75.74	0.58	6.11	9.57	30.63
Ours	75.97	0.57	6.05	6.84	24.72
MBv2-1.0×	72.92	0.30	3.50	6.68	20.82
Ours	73.42	0.29	3.31	4.82	16.61
MBv2-0.7×	68.98	0.17	2.48	5.26	15.81
Ours	69.16	0.15	1.81	3.39	11.62
MBv2-0.5×	64.70	0.10	1.97	3.64	10.97
Ours	64.68	0.09	1.17	2.59	8.51

4.2. Compare with SoTAs

To compare with previous state-of-the-arts, we conduct extensive experiments of image classification on ImageNet

Table 3: Prune RegNetX on ImageNet.

model	T1 (%)	F (10^9)	P (10^6)	M (10^6)	T (ms)
RegX-3.2G	78.72	3.176	15.29	11.36	65.99
Ours	78.80	3.228	14.34	9.89	61.34
RegX-1.6G	77.29	1.602	9.19	7.93	41.21
Ours	77.97	1.588	9.30	7.29	40.44
RegX-0.8G	75.24	0.799	7.26	5.15	26.71
Ours	76.39	0.799	5.93	5.38	25.51
RegX-0.4G	72.28	0.398	5.16	3.14	16.32
Ours	73.75	0.399	5.42	2.78	14.97
RegX-0.2G	68.65	0.199	2.68	2.16	10.09
Ours	70.01	0.199	2.77	2.13	11.49

Table 4: Compare with SoTAs on ImageNet. The **column** "T1" represents top-1 accuracy of the pruned model on the validation set where ↓ shows the accuracy drop compared with the unpruned model. "B1" shows the top-1 accuracy of the unpruned base model. "F" shows the amount of FLOPs of the pruned model, where ↓ elements show the relative FLOPs reduction compared with the unpruned model. "S" denotes the actual speedup of the pruned model on GPUs.

	method	T1(%)	B1(%)	F(G)	S(×)
Res50	ThiNet (Luo et al., 2017)	74.03	75.30	2.58	1.13
	SSS (Huang & Wang, 2018)	75.44	76.12	3.47	-
	IE (Molchanov et al., 2019)	76.43	76.18	3.27	-
	HetConv (Singh et al., 2020)	76.16	76.16	2.85	-
	Meta (Liu et al., 2019)	76.20	76.60	3.0	-
	GBN (You et al., 2019)	76.19	75.85	2.43	-
	Ours	76.95	76.79	3.06	1.30
	ThiNet (Luo et al., 2017)	72.03	75.30	1.81	1.27
	CP (He et al., 2017)	75.06	76.13	2.04	-
	NISP (Yu et al., 2018)	0.89↓	-	2.29	-
	SFP (He et al., 2018a)	74.61	76.15	2.38	1.43
	GDP (Lin et al., 2018)	72.61	75.13	2.24	1.24
	SSS (Huang & Wang, 2018)	71.82	76.12	2.33	-
	DCP (Zhuang et al., 2018)	74.95	76.01	1.81	-
	AOFP (Ding et al., 2019b)	75.11	75.34	1.77	-
	FPGM (He et al., 2019)	74.83	76.15	1.90	1.62
	IE (Molchanov et al., 2019)	74.50	76.18	2.25	-
	C-SGD (Ding et al., 2019a)	74.54	75.33	2.20	-
	Meta (Liu et al., 2019)	75.40	76.60	2.0	-
	GBN (You et al., 2019)	75.18	75.85	1.84	-
	LFPC (He et al., 2020)	74.46	76.15	1.60	-
	HRank (Lin et al., 2020)	74.98	76.15	2.30	-
	Ours	76.42	76.79	2.04	1.79
	ThiNet (Luo et al., 2017)	68.17	75.30	1.17	1.35
	GDP (Lin et al., 2018)	70.93	75.13	1.57	-
	IE (Molchanov et al., 2019)	71.69	76.18	1.34	-
	Meta (Liu et al., 2019)	73.40	76.60	1.0	-
	CURL (Luo & Wu, 2020)	73.39	76.15	1.11	-
	Ours	73.94	76.79	1.02	2.94
Res101	ISTA (Ye et al., 2018)	75.27	76.40	4.47	-
	SFP (He et al., 2018a)	77.51	77.37	4.51	-
	SSS (Huang & Wang, 2018)	75.44	76.40	3.47	-
	AOFP (Ding et al., 2019b)	76.40	76.63	3.89	-
	FPGM (He et al., 2019)	77.32	77.37	4.51	-
	IE (Molchanov et al., 2019)	77.35	77.37	4.70	-
	Ours	78.33	78.29	3.90	1.50
MBv2	AMC (He et al., 2018b)	70.80	71.80	0.22	-
	Meta (Liu et al., 2019)	72.70	74.70	0.29	-
	Ours	73.42	75.74	0.29	1.84
	Meta (Liu et al., 2019)	68.20	74.70	0.14	-
	Ours	69.16	75.74	0.15	1.79
	Meta (Liu et al., 2019)	65.00	74.70	0.11	-
	Ours	65.94	75.74	0.10	1.82
NeXt50	SSS (Huang & Wang, 2018)	74.98	77.57	2.43	-
	Ours	77.53	77.97	2.11	1.57

Table 5: Prune detection networks including RetinaNet, FSAF, ATSS, PAA and Faster R-CNN on COCO. The "AP" column shows the mAP (%) on the validation set and the "T" column is the inference time. The other columns and the suffix in rows have the same meaning as Tab. 1 (* : no Group Normalization in heads to compare with RetinaNet).

model	AP	F(10^9)	P(10^6)	M(10^6)	T(ms)	S(×)
Retina	36.5	238.5	37.96	297.4	39.73	-
Retina-I	36.3	119.2	30.31	246.1	28.49	1.39
Retina-M	36.5	119.2	26.34	190.2	25.36	1.57
Retina-F	36.8	119.2	26.18	254.1	30.08	1.32
Retina-U	35.8	119.2	14.18	244.2	28.53	1.39
FSAF	37.4	205.5	36.41	283.1	37.10	-
FSAF-I	37.2	102.7	28.52	231.5	26.89	1.38
FSAF-M	37.3	102.7	24.09	171.3	23.00	1.61
FSAF-F	37.6	102.7	23.19	233.6	27.90	1.33
FSAF-U	36.4	102.7	19.23	242.2	27.99	1.33
ATSS*	38.1	204.4	32.29	283.1	37.14	-
ATSS-I	37.9	102.2	24.48	234.6	26.76	1.39
ATSS-M	38.0	102.2	22.20	177.1	23.72	1.57
ATSS-F	38.3	102.2	21.54	236.9	28.18	1.32
ATSS-U	36.7	102.1	12.41	228.5	26.35	1.41
PAA*	39.0	204.4	32.29	283.1	37.16	-
PAA-I	39.4	102.2	24.82	234.7	27.29	1.36
PAA-M	39.4	102.1	23.00	174.9	23.30	1.59
PAA-F	39.6	102.1	21.95	235.5	27.81	1.34
PAA-U	38.5	102.2	13.19	228.4	26.39	1.41
Faster	37.4	199.3	41.75	294.6	44.28	-
Faster-M	37.8	99.55	30.96	197.0	25.58	1.73
Faster-M	36.6	49.82	17.48	130.8	14.43	3.07
Faster-F	37.8	99.70	25.95	240.6	28.79	1.54
Faster-U	33.5	99.90	9.861	224.8	28.06	1.58

using different network structures and FLOPs constraints. From Fig. 1 and Tab. 4 we can see that our method performs best. Specifically, we outperform layer-wise pruning methods such as CP (He et al., 2017) and ThiNet (Luo et al., 2017) because we evaluate the importance scores globally throughout the network. Moreover, we do not need sensitivity analysis which is required by NISP (Yu et al., 2018) to decide the pruning ratio for each layer, as our method can automatically learn to prune the least important channels considering the current state of the network. We also achieve better accuracy than the methods C-SGD (Ding et al., 2019a), GBN (You et al., 2019) and IE (Molchanov et al., 2019) which compute the overall importance of coupled channels via heuristics, validating the benefits of our importance metric grounded on gradients obtained by the principled chain rule.

4.3. Prune for Detection

Pruning object detection is more challenging than image classification due to its larger input size and more complicated networks, which demands model pruning more than image classification. Besides, many pruning methods based on BN scaling parameters can not be directly applied. However, our method can not only be applied to image classification, but also object detection, thanks to its general importance estimation and the proposed layer grouping for pruning coupled channels. We prune one-stage meth-

Table 6: Compare with Slimmable Networks for pruning Faster-RCNN on COCO. AP for pruned model and the unpruned model (B-AP), and the AP drop (△) are shown. "F" denotes the percentage of remained FLOPs.

model	AP(%)	B-AP(%)	△(%)	F(%)
Slim (Yu et al., 2019)	36.1	36.4	0.3 ↓	56
Ours	37.8	37.4	0.4 ↑	50
Slim (Yu et al., 2019)	34.0	36.4	2.4 ↓	25
Ours	36.6	37.4	0.8 ↓	25

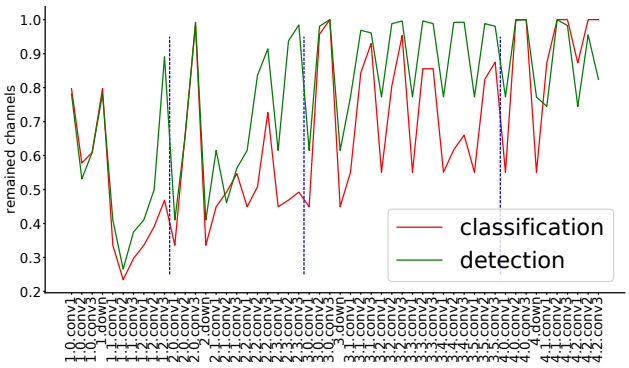


Figure 7: The percentage of remained channels in the pruned backbone over different layers for classification and detection. Features maps are 2× down-sampled at blue dashed lines.

ods including RetinaNet (Lin et al., 2017b), FSAF (Zhu et al., 2019), ATSS (Zhang et al., 2020) and PAA (Kim & Lee, 2020), and two-stage method Faster R-CNN (Ren et al., 2015) to extensively validate the effectiveness of our method. We present the pruning results of our method for various detection frameworks in Fig. 2 (d) and Tab. 5. Similar to pruning classification models, normalizing importances with memory reduction and pruning coupled channels together ("-M") leads to the highest efficiency. Our method effectively prunes detection networks without losing average precision, in some cases our pruned model even receives higher mAP than the unpruned baseline. More importantly, our method delivers practical inference speedup, *e.g.*, we achieve a 3× speedup by pruning Faster R-CNN with only 0.8% mAP drop. We also compare the pruned networks with state-of-the-art method Slimmable Networks (Yu et al., 2019) in Tab. 6, as shown we can achieve higher mAP, lower mAP drop under comparable or less FLOPs.

Considering the intrinsic differences between image classification and object detection, we compare the pruned network structures between them. As in Fig. 7, we find that the pruned classification network keeps more capacity in later stages where the spatial resolution is rather small, as classification needs more global features. However, for detection the early stages also remain a large portion of channels, as detection should extract features at different scales to detect objects with various sizes. This validates that our method can be adaptively applied to different tasks.

5. Conclusion

We present a general channel pruning framework for complicated structures. We propose the layer grouping algorithm to find coupled channels and make them share the binary mask. Based on the single-channel importance approximated by Fisher information, we compute the overall importance of coupled channels by the chain rule of gradient computation. We prune the coupled channels simultaneously for better accuracy-efficiency trade-off. Moreover, normalizing channel importances by memory reduction rather than FLOPs is proposed to deliver more speedup. Extensive experiments on pruning various network structures with residual connections, GConv/DWConv and FPN in detection are explored and verify the effectiveness.

Inspired by the memory-bound nature of GPUs, we propose to normalize channel importance by memory reduction, which can bring a better trade-off between accuracy and speedup. In future work we will theoretically model the relationships between core factors (*e.g.,* FLOPs, memory) and inference speed on various platforms (*e.g.,* GPU/CPU/TPU).

Acknowledgements

The work described in this paper was partially supported by the Natural Science Foundation of Guangdong Province (No. 2020A1515010711), the Special Foundation for the Development of Strategic Emerging Industries of Shenzhen (No. JCYJ20200109143010272 and No. JCYJ20200109143035495), the Innovation and Technology Commission of the Hong Kong Special Administrative Region, China (Enterprise Support Scheme under the Innovation and Technology Fund B/E030/18) and the Shanghai Committee of Science and Technology, China (Grant No. 20DZ1100800).

References

Deng, J., Dong, W., Socher, R., Li, L., Kai Li, and Li Fei-Fei. Imagenet: A large-scale hierarchical image database. In *2009 IEEE Conference on Computer Vision and Pattern Recognition*, pp. 248–255, 2009. 6

Ding, X., Ding, G., Guo, Y., and Han, J. Centripetal sgd for pruning very deep convolutional networks with complicated structure. In *Proceedings of the IEEE Conference on Computer Vision and Pattern Recognition*, pp. 4943–4953, 2019a. 8

Ding, X., Ding, G., Guo, Y., Han, J., and Yan, C. Approximated oracle filter pruning for destructive cnn width

optimization. In *International Conference on Machine Learning*, 2019b. 8

Dong, X., Chen, S., and Pan, S. Learning to prune deep neural networks via layer-wise optimal brain surgeon. In *Advances in Neural Information Processing Systems*, pp. 4857–4867, 2017. 3

Guo, Y., Yao, A., and Chen, Y. Dynamic network surgery for efficient dnns. In *Advances in Neural Information Processing Systems*, pp. 1379–1387, 2016. 2

Han, S., Pool, J., Tran, J., and Dally, W. Learning both weights and connections for efficient neural network. In *Advances in Neural Information Processing Systems*, pp. 1135–1143, 2015. 2

Han, S., Mao, H., and Dally, W. J. Deep compression: Compressing deep neural networks with pruning, trained quantization and huffman coding. *International Conference on Learning Representations (ICLR)*, 2016. 2

Hassibi, B. and Stork, D. G. Second order derivatives for network pruning: Optimal brain surgeon. In *Advances in Neural Information Processing Systems*, pp. 164–171, 1993. 3

He, K., Zhang, X., Ren, S., and Sun, J. Deep residual learning for image recognition. In *2016 IEEE Conference on Computer Vision and Pattern Recognition (CVPR)*, pp. 770–778, 2016. 1, 2, 5, 6

He, Y., Zhang, X., and Sun, J. Channel pruning for accelerating very deep neural networks. In *Proceedings of the IEEE International Conference on Computer Vision*, pp. 1389–1397, 2017. 1, 2, 3, 8

He, Y., Kang, G., Dong, X., Fu, Y., and Yang, Y. Soft filter pruning for accelerating deep convolutional neural networks. In *International Joint Conference on Artificial Intelligence (IJCAI)*, pp. 2234–2240, 2018a. 3, 8

He, Y., Lin, J., Liu, Z., Wang, H., Li, L.-J., and Han, S. Amc: Automl for model compression and acceleration on mobile devices. In *Proceedings of the European Conference on Computer Vision (ECCV)*, pp. 784–800, 2018b. 8

He, Y., Liu, P., Wang, Z., Hu, Z., and Yang, Y. Filter pruning via geometric median for deep convolutional neural networks acceleration. In *Proceedings of the IEEE Conference on Computer Vision and Pattern Recognition*, pp. 4340–4349, 2019. 8

He, Y., Ding, Y., Liu, P., Zhu, L., Zhang, H., and Yang, Y. Learning filter pruning criteria for deep convolutional neural networks acceleration. In *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition*, pp. 2009–2018, 2020. 8

Huang, Z. and Wang, N. Data-driven sparse structure selection for deep neural networks. In *Proceedings of the European Conference on Computer Vision (ECCV)*, pp. 304–320, 2018. 3, 8

Kim, K. and Lee, H. S. Probabilistic anchor assignment with iou prediction for object detection. In *Proceedings of the European Conference on Computer Vision (ECCV)*, 2020. 9

Krizhevsky, A., Sutskever, I., and Hinton, G. E. Imagenet classification with deep convolutional neural networks. In *Advances in Neural Information Processing Systems*, pp. 1097–1105, 2012. 1, 5

Lebedev, V. and Lempitsky, V. Fast convnets using group-wise brain damage. In *Proceedings of the IEEE Conference on Computer Vision and Pattern Recognition*, pp. 2554–2564, 2016. 3

LeCun, Y., Denker, J. S., and Solla, S. A. Optimal brain damage. In *Advances in Neural Information Processing Systems*, pp. 598–605, 1990. 3

Li, C., Dakkak, A., Xiong, J., and Hwu, W. Benanza: Automatic ubenchmark generation to compute "lower-bound" latency and inform optimizations of deep learning models on gpus. In *2020 IEEE International Parallel and Distributed Processing Symposium (IPDPS)*, pp. 440–450, 2020. 6

Li, H., Kadav, A., Durdanovic, I., Samet, H., and Graf, H. P. Pruning filters for efficient convnets. *International Conference on Learning Representations (ICLR)*, 2017. 3

Lin, M., Ji, R., Wang, Y., Zhang, Y., Zhang, B., Tian, Y., and Shao, L. Hrank: Filter pruning using high-rank feature map. In *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition*, pp. 1529–1538, 2020. 8

Lin, S., Ji, R., Li, Y., Wu, Y., Huang, F., and Zhang, B. Accelerating convolutional networks via global & dynamic filter pruning. In *International Joint Conference on Artificial Intelligence (IJCAI)*, pp. 2425–2432, 2018. 8

Lin, T.-Y., Maire, M., Belongie, S., Hays, J., Perona, P., Ramanan, D., Dollár, P., and Zitnick, C. L. Microsoft coco: Common objects in context. In *Proceedings of the European Conference on Computer Vision (ECCV)*, pp. 740–755, 2014. 6

Lin, T.-Y., Dollár, P., Girshick, R., He, K., Hariharan, B., and Belongie, S. Feature pyramid networks for object detection. In *Proceedings of the IEEE Conference on Computer Vision and Pattern Recognition*, pp. 2117–2125, 2017a. 1, 5

Lin, T.-Y., Goyal, P., Girshick, R., He, K., and Dollár, P. Focal loss for dense object detection. In *Proceedings of the IEEE International Conference on Computer Vision*, pp. 2980–2988, 2017b. 9

Liu, Z., Li, J., Shen, Z., Huang, G., Yan, S., and Zhang, C. Learning efficient convolutional networks through network slimming. In *Proceedings of the IEEE International Conference on Computer Vision*, pp. 2736–2744, 2017. 1, 2, 3

Liu, Z., Mu, H., Zhang, X., Guo, Z., Yang, X., Cheng, K.-T., and Sun, J. Metapruning: Meta learning for automatic neural network channel pruning. In *Proceedings of the IEEE International Conference on Computer Vision*, pp. 3296–3305, 2019. 8

Luo, J.-H. and Wu, J. Neural network pruning with residual-connections and limited-data. In *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition*, pp. 1458–1467, 2020. 2, 3, 8

Luo, J.-H., Wu, J., and Lin, W. Thinet: A filter level pruning method for deep neural network compression. In *Proceedings of the IEEE International Conference on Computer Vision*, pp. 5058–5066, 2017. 1, 2, 3, 8

Mishra, A. K., Latorre, J. A., Pool, J., Stosic, D., Stosic, D., Venkatesh, G., Yu, C., and Micikevicius, P. Accelerating sparse deep neural networks. *arXiv preprint arXiv:2104.08378*, 2021. 2

Molchanov, P., Tyree, S., Karras, T., Aila, T., and Kautz, J. Pruning convolutional neural networks for resource efficient inference. *International Conference on Learning Representations (ICLR)*, 2017. 1, 3

Molchanov, P., Mallya, A., Tyree, S., Frosio, I., and Kautz, J. Importance estimation for neural network pruning. In *Proceedings of the IEEE Conference on Computer Vision and Pattern Recognition*, pp. 11264–11272, 2019. 3, 8

Paszke, A., Gross, S., Massa, F., Lerer, A., Bradbury, J., Chanan, G., Killeen, T., Lin, Z., Gimelshein, N., Antiga, L., et al. Pytorch: An imperative style, high-performance deep learning library. In *Advances in Neural Information Processing Systems*, pp. 8024–8035, 2019. 7

Radosavovic, I., Kosaraju, R. P., Girshick, R., He, K., and Dollár, P. Designing network design spaces. In *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition*, pp. 10428–10436, 2020. 1, 2, 6

Ren, S., He, K., Girshick, R., and Sun, J. Faster r-cnn: Towards real-time object detection with region proposal networks. In *Advances in Neural Information Processing Systems*, pp. 91–99, 2015. 9

Sandler, M., Howard, A., Zhu, M., Zhmoginov, A., and Chen, L.-C. Mobilenetv2: Inverted residuals and linear bottlenecks. In *Proceedings of the IEEE Conference on Computer Vision and Pattern Recognition*, pp. 4510–4520, 2018. 1, 2, 5, 6

Simonyan, K. and Zisserman, A. Very deep convolutional networks for large-scale image recognition. In *International Conference on Learning Representations*, 2015. 1, 5

Singh, P., Verma, V. K., Rai, P., and Namboodiri, V. P. Hetconv: Beyond homogeneous convolution kernels for deep cnns. *International Journal of Computer Vision.*, 128(8):2068–2088, 2020. 8

Singh, S. P. and Alistarh, D. Woodfisher: Efficient second-order approximation for neural network compression. In *Advances in Neural Information Processing Systems*, pp. 18098–18109, 2020. 3

Theis, L., Korshunova, I., Tejani, A., and Huszár, F. Faster gaze prediction with dense networks and fisher pruning. *arXiv preprint arXiv:1801.05787*, 2018. 3, 4

Wang, H., Wu, Z., Liu, Z., Cai, H., Zhu, L., Gan, C., and Han, S. HAT: Hardware-aware transformers for efficient natural language processing. In *Proceedings of the 58th Annual Meeting of the Association for Computational Linguistics*, pp. 7675–7688, 2020. 6

Wen, W., Wu, C., Wang, Y., Chen, Y., and Li, H. Learning structured sparsity in deep neural networks. In *Advances in Neural Information Processing Systems*, pp. 2074–2082, 2016. 3

Xie, S., Girshick, R., Dollár, P., Tu, Z., and He, K. Aggregated residual transformations for deep neural networks. In *Proceedings of the IEEE Conference on Computer Vision and Pattern Recognition*, pp. 1492–1500, 2017. 1, 2, 5, 6

Ye, J., Lu, X., Lin, Z., and Wang, J. Z. Rethinking the smaller-norm-less-informative assumption in channel pruning of convolution layers. In *International Conference on Learning Representations (ICLR)*, 2018. 3, 8

You, Z., Yan, K., Ye, J., Ma, M., and Wang, P. Gate decorator: Global filter pruning method for accelerating deep convolutional neural networks. In *Advances in Neural Information Processing Systems*, pp. 2133–2144, 2019. 8

Yu, J., Yang, L., Xu, N., Yang, J., and Huang, T. Slimmable neural networks. In *International Conference on Learning Representations*, 2019. 9

Yu, R., Li, A., Chen, C.-F., Lai, J.-H., Morariu, V. I., Han, X., Gao, M., Lin, C.-Y., and Davis, L. S. Nisp: Pruning

networks using neuron importance score propagation. In *Proceedings of the IEEE Conference on Computer Vision and Pattern Recognition*, pp. 9194–9203, 2018. 2, 3, 8

Zhang, S., Chi, C., Yao, Y., Lei, Z., and Li, S. Z. Bridging the gap between anchor-based and anchor-free detection via adaptive training sample selection. In *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition*, pp. 9759–9768, 2020. 9

Zhou, A., Ma, Y., Zhu, J., Liu, J., Zhang, Z., Yuan, K., Sun, W., and Li, H. Learning n:m fine-grained structured sparse neural networks from scratch. In *International Conference on Learning Representations*, 2021. 2

Zhu, C., He, Y., and Savvides, M. Feature selective anchor-free module for single-shot object detection. In *Proceedings of the IEEE Conference on Computer Vision and Pattern Recognition*, pp. 840–849, 2019. 9

Zhuang, Z., Tan, M., Zhuang, B., Liu, J., Guo, Y., Wu, Q., Huang, J., and Zhu, J. Discrimination-aware channel pruning for deep neural networks. In *Advances in Neural Information Processing Systems*, pp. 875–886, 2018. 8